

CS 250 Final Project

Luis Conan Losch Vargas

South New Hampshire University

CS-250- Software Development Lifecycle

Professor Robin Chataut

June 16, 2026

Sprint Review and Retrospective: SNHU Travel Project

Introduction

As a member of the ChadaTech development team piloting the Scrum-Agile methodology for the SNHU Travel application, I had the opportunity to rotate through various roles, including Scrum Master, Developer, Product Owner, and Product Tester. This retrospective summarizes our sprint execution, evaluates our organizational tools, and assesses the overall effectiveness of the Agile approach for this specific project.

Applying Roles

The success of the SNHU Travel project relied heavily on the distinct, collaborative contributions of our Scrum-Agile team roles. As the **Scrum Master**, my primary contribution was facilitating Scrum events and removing impediments through servant leadership rather than directive authority. For example, when our development team was blocked by a complex flight booking API integration, I ensured the issue was visibly flagged so the team could "swarm" the problem and resolve it collaboratively.

The **Product Owner (PO)** contributed by acting as the bridge between stakeholders and the development team. They ensured our backlog was grounded in real-world problems, such as uncovering traveler anxiety over hidden fees and the need for flexible cancellation policies during stakeholder interviews. The **Development Team** (which included my role as a **Developer** and **Product Tester**) contributed by self-organizing to execute the technical work while advocating for quality. For instance, as a tester, I proactively collaborated with the PO to ensure that non-functional requirements, such as

WCAG 2.2 AA accessibility standards and API timeout error handling, were not overlooked, ensuring a robust, inclusive product.

Completing User Stories

A Scrum-Agile approach was instrumental in bringing user stories to completion by breaking large, complex features into manageable, iterative tasks with clear acceptance criteria. A specific example was the user story: *"As a family traveler, I want to filter lodging by proximity to attractions and included breakfast so that I can quickly find suitable vacation options."*

By framing the requirement in this standard format, the team could focus on user value rather than just technical implementation. During Sprint Planning, we broke this down into smaller technical tasks (e.g., updating the database query, modifying the front-end UI, writing test cases). Because the PO provided clear, measurable acceptance criteria, I, as a tester, could directly derive positive test paths, and as a developer, I knew exactly what constituted the "Definition of Done." This iterative breakdown ensured continuous progress and prevented the team from being overwhelmed by the scope of the feature.

Handling Interruptions

The true test of our Scrum-Agile approach occurred when we realized mid-sprint that some user stories lacked explicit non-functional requirements, such as maximum acceptable response times for flight searches on constrained 3G networks or detailed error-handling protocols for third-party API failures. In a rigid methodology, discovering this during development would have caused significant rework or delayed the release. However, the Scrum-Agile approach supported us by providing structured feedback loops. Instead of derailing the current sprint, I formally requested clarification from the Product Owner via a targeted communication. We documented these missing elements and added them to the Product

Backlog. During the next Backlog Refinement session, we prioritized these non-functional criteria for the upcoming sprint, ensuring that quality was "built in" rather than "inspected in" later, without breaking the team's focus on the current sprint goal.

Communication Effective communication is the lifeblood of Agile, creating the transparency necessary for inspection and adaptation. Below are samples of communication used during the project:

Sample 1: Developer to Product Owner/Tester (Regarding User Story #42)

"Hi Jane and John, As I begin development on User Story #42 (Secure User Login) for our current sprint, I have encountered a minor blocker regarding the password reset functionality. The current acceptance criteria do not specify whether the reset link should expire after 15 minutes or 24 hours, nor does it detail the required error message if a user attempts to use an expired link. Could you please provide clarification on these two points by end-of-day tomorrow? Please reply directly to this thread or let me know if a quick 5-minute sync during tomorrow's Daily Scrum would be more efficient."

Sample 2: Product Tester to Product Owner (Regarding Non-Functional Requirements)

"As I finalize the test cases for the upcoming sprint's travel booking features, I need a few clarifications to ensure our test coverage aligns with quality standards. Could you please provide: 1) The maximum acceptable response time for search results on mobile devices under 3G/4G conditions, 2) Confirmation of the target accessibility compliance level (e.g., WCAG 2.2 AA), and 3) Preferred user-facing error messages when the airline reservation API times out."

Why these were effective:

Both samples were highly effective because they were proactive, specific, and time-bound. They avoided lengthy, ambiguous email chains by clearly stating the blocker, the exact information needed, and a

proposed solution (e.g., a 5-minute sync). This encouraged collaboration by respecting the stakeholders' time while ensuring the development and testing processes could move forward without guesswork.

Organizational Tools

Our team utilized **Jira** as our primary organizational tool, which functioned as a digital "information radiator" to sustain transparency. Jira was deeply integrated with our Scrum events. During the **Daily Standup**, we used the Jira board to visually flag blockers, such as the aforementioned flight booking API issue, which immediately prompted the team to swarm the problem.

Furthermore, Jira increased our efficiency by centralizing communication and automating workflows. When a developer moved a user story to the "Done" column, the tool automatically updated the sprint burndown chart and notified the Product Owner. This eliminated the need for manual status-reporting meetings. During our **Sprint Retrospectives**, we utilized structured formats (such as the "Sailboat" or "Mad/Sad/Glad" techniques) documented within our project space to evaluate our processes, ensuring that actionable improvements were tracked and implemented in the next sprint.

Evaluating Agile Process

Pros and Cons of the Scrum-Agile Approach:

The primary pro of using Scrum-Agile for the SNHU Travel project was adaptability. Iterative stakeholder collaboration significantly improved requirement accuracy and allowed us to pivot quickly when user preferences shifted (e.g., adding flexible cancellation policies). It also fostered a supportive environment where transparency reduced team stress. However, a con was the initial learning curve required to write highly refined user stories with both functional and non-functional acceptance criteria. Additionally, the

time commitment for Scrum ceremonies initially felt like an administrative adjustment for team members used to traditional workflows.

Was it the best approach?

Yes, the Scrum-Agile approach was definitively the best approach for the SNHU Travel development project. The travel technology space is highly competitive, and user preferences evolve rapidly. A rigid, linear plan would have risked delivering an outdated product or missing critical usability factors like accessibility. Agile allowed us to deliver a Minimum Viable Product (MVP) quickly, incorporate continuous testing, and iterate based on real user feedback, ultimately providing a much higher, more resilient value to the client.